西南交通大学
Southwest Jiaotong University

《网络空间安全技术》课程设计报告

设计题目：Web 登录系统安全加固与审计日志设计

学　　号：2023112621

姓　　名：程泽凯

班　　级：计算机类 2023 级

指导教师：XXX　老师

2026 年 5 月 27 日

《视觉计算》课程设计任务书

设计题目	**Web** 登录系统安全加固与审计日志设计	成绩	
课程设计主要内容	针对计算机视觉领域某一特定问题，例如：目标检测、人脸识别、图像分割、图像生成等，查阅国内外近 5 年来相关技术资料，选择 3 篇以上具有代表性论文进行深入阅读，分析并理解其原理和方法。撰写一篇不少于 4000 字的课程设计报告，对相关论文的理论原理及算法进行阐述，对其中的方法动手搭建模型（建议基于 MindSpore），编写程序复现实验结果，进行对比分析，给出结论。鼓励提出一定的改良措施，完成实验（建议基于华为 MindSpore）。 课程设计以组队的形成开展，每组 2-3 人，自行组队，报告中明确每个人的分工。报告使用老师提供的 latex 模版，编译方式选 XeLatex。模版已写好第一至第六章的目录，尽量按照该目录撰写报告。		
指导教师评语	建议：从学生的工作态度、工作量、设计（论文）的创造性、学术性、实用性及书面表达能力等方面给出评价。 签名：　　年　　月　　日		

目录

一、合规声明与个人分工 **1**

二、项目背景 **1**

三、实验环境 **1**

四、需求分析 **2**

4.1 功能需求 2

4.2 安全需求 2

4.3 异常规则需求与个性化阈值设计 3

五、技术方案 **3**

5.1 系统总体架构 3

5.2 密码哈希方案 3

5.3 数据库设计 4

5.4 登录失败限制与异常检测策略 5

六、系统流程图 **5**

6.1 系统总体流程图 5

6.2 用户登录认证流程图（核心） 5

6.3 登录失败限制流程图 6

6.4 异常登录审计流程图 6

七、系统实现 **7**

7.1 登录认证核心 7

7.2 异常检测实现 8

八、实验数据 **9**

8.1 测试账号（本人创建） 9

8.2 模拟异常登录记录（本人设计） 9

8.3 审计真实输出 10

九、测试用例与运行结果 **11**

9.1 测试用例 11

9.2 运行结果 11

十、问题与改进 **11**

10.1遇到的问题及解决 11
10.2改进方向 12
10.3答辩可现场验证的参数 12
十一总结 **12**
参考文献 **12**

一、 合规声明与个人分工

合规声明：本课程设计全部在本机（Windows 11 家庭中文版 + Python 3.13）本地离线环境与本地浏览器中完成与测试。数据库、登录日志、测试账号与异常登录记录均为本人在离线环境中自行生成的模拟数据，**未对任何真实第三方系统、公网服务器或校园网设备进行扫描、攻击或测试，未造成任何影响。**

完成方式：本题为单人完成。全部工作由本人（程泽凯，学号 2023112621，学号尾号 1）独立完成，包括：需求分析与安全规则设计、系统编码实现（注册登录、口令哈希、失败限制、审计日志、异常检测）、测试用例编写与运行、流程图绘制以及报告撰写与 LaTeX 排版。

二、 项目背景

登录系统是 Web 应用的安全入口，也是攻击者的首要目标。一旦登录环节被攻破，攻击者即可获得合法身份并进一步访问受保护资源。常见的登录安全威胁包括：

- **口令明文或弱哈希存储：**数据库一旦泄露，口令直接暴露，并可能被用于撞库攻击；
- **暴力破解与撞库：**攻击者借助自动化脚本对账号进行高频口令尝试；
- **异常登录缺乏感知：**异地登录、非工作时间登录、高频请求等可疑行为得不到及时识别；
- **缺乏审计能力：**登录行为无日志记录，安全事件发生后无法追溯定位。

针对上述问题，OWASP 在其认证安全指南中明确建议对口令进行加盐慢哈希存储、对失败登录进行限制并记录审计日志 [4]；NIST SP 800-63B 数字身份指南同样强调了口令存储与认证生命周期管理的安全要求 [1]。本课程设计正是围绕这些安全实践，构建一个具备**安全加固与审计能力**的小型登录系统，做到“有场景、有数据、有流程、有规则、有实现、有测试、有结果、有分析”。

三、 实验环境

本系统的开发与测试环境如表 1 所示。选型上采用 Python Flask + SQLite，原因在于二者轻量、零外部服务依赖，便于本地一键运行与答辩现场演示；同时不引入 ORM，所有 SQL 透明可审计，便于讲解与安全分析。

表 **1**　实验环境配置

类别	配置
操作系统	Windows 11 家庭中文版（10.0.26200）
开发语言	Python 3.13.5
Web 框架	Flask 3.1.0 / Werkzeug 3.1.3
数据库	SQLite 3（Python 标准库 `sqlite3`）
口令哈希	PBKDF2-SHA256，26 万次迭代，自带随机盐
运行与测试	本地浏览器（127.0.0.1:5000）+ 自动化测试脚本

四、 需求分析

4.1 功能需求

任务书要求“必须实现的功能”共 7 项，本系统逐条实现并定位到对应代码模块，如表 2 所示。

表 **2**　必做功能与实现位置对照

序号	必须实现的功能	实现位置
1	用户注册	`app.py` / `security.register_user`
2	用户登录	`app.py` / `security.authenticate`
3	密码哈希存储	`security.hash_password`
4	登录失败计数	`security.count_recent_failures`
5	账号临时锁定	`security` 锁定逻辑 + `locked_until`
6	登录日志记录	`db.add_login_log` + `login_logs` 表
7	异常登录提醒	`audit.detect_realtime`

4.2 安全需求

- 口令绝不明文存储，采用加盐慢哈希；

- 用户名不存在与口令错误返回统一提示，防止用户名枚举；
- 所有数据库操作使用参数化查询（占位符 ?），防止 SQL 注入；
- 失败登录达阈值后临时锁定账号，抵抗暴力破解；
- 审计可重复执行并输出结构化结果，便于事后追溯。

4.3 异常规则需求与个性化阈值设计

任务书给出四类异常的规则示例，本人结合自身上机作息与实验场景，将其细化为可量化、可现场修改的阈值，如表 3 所示。所有阈值集中存放于 `config.py`，答辩时可现场修改任一数值并重新运行验证。

表 **3**　四类异常的个性化判断规则（设计者：程泽凯）

异常类型	任务书规则示例	本人设计的判断规则
暴力尝试	5 分钟内失败超过 5 次	5 分钟窗口内失败 ≥ 5 次 $\rightarrow$ 锁定 10 分钟
异地登录	同账号短时不同 IP	10 分钟内同账号出现 ≥ 2 个不同 IP
非工作时间	晚 23:00 后登录	23:00–06:00 之间成功登录
高频登录	1 分钟内超过阈值	60 秒内请求 ≥ 10 次

五、 技术方案

5.1 系统总体架构

系统采用分层设计，由配置层、数据访问层、安全核心层、审计层与 Web 表现层构成。各模块职责单一、低耦合，所有安全阈值集中于配置层。整体调用关系如下：浏览器经 HTTP 访问 Flask 路由层（`app.py`），路由层调用安全核心（`security.py`）完成认证，调用审计模块（`audit.py`）完成异常检测，二者均通过数据访问层（`db.py`）读写 SQLite 数据库。

5.2 密码哈希方案

系统采用 Werkzeug 提供的 `pbkdf2:sha256`（26 万次迭代）对口令进行加盐慢哈希存储，符合 PKCS #5 / RFC 8018 对基于口令的密钥派生函数的规范 [2]。其安全意义在于三点：（1）**加盐**——每个口令使用独立随机盐，相同口令的哈希值各不相同，可抵抗

彩虹表攻击；（2）**慢哈希**——26 万次迭代显著抬高离线爆破成本；（3）**恒定时间比较**——校验时使用恒定时间比较，防止时序侧信道泄露。核心实现如代码清单 1 所示。

```python
from werkzeug.security import generate_password_hash, check_password_hash

# Store: salted slow hash, never plaintext
def hash_password(plain):
    return generate_password_hash(plain, method="pbkdf2:sha256:260000")

# Verify: constant-time comparison
def verify_password(stored_hash, plain):
    return check_password_hash(stored_hash, plain)
```

Listing 1: 口令哈希与校验（security.py 节选）

5.3 数据库设计

数据库包含三张表:用户表 `users`、登录日志表 `login_logs` 与审计告警表 `audit_alerts`，并为日志的高频查询字段建立联合索引，如代码清单 2 所示。其中 `login_logs.reason` 取值为 `ok / wrong_password / user_not_found / locked`，是后续审计判断的关键字段。

```sql
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at TEXT NOT NULL,
  failed_count INTEGER NOT NULL DEFAULT 0,
  locked_until TEXT,       -- lock deadline, NULL = not locked
  last_login_at TEXT
);
CREATE TABLE login_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL, ip TEXT NOT NULL, user_agent TEXT,
  success INTEGER NOT NULL, -- 1 success / 0 failure
  reason TEXT NOT NULL, created_at TEXT NOT NULL
);
CREATE TABLE audit_alerts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL, alert_type TEXT NOT NULL,
  detail TEXT NOT NULL, created_at TEXT NOT NULL
);
CREATE INDEX idx_logs_user_time ON login_logs(username, created_at);
```

Listing 2: 数据库表结构（db.py 节选）

5.4 登录失败限制与异常检测策略

失败计数采用**基于日志的时间窗口统计**而非简单计数器累加，因此“5 分钟内失败 5 次”这一时间语义是精确的。当窗口内失败次数达到阈值，即写入锁定截止时间并生成暴力尝试告警。异常检测分为两个阶段：登录成功瞬间运行轻量的**实时检测**（异地 / 非工作时间 / 高频），命中即弹出提醒；离线运行**全量审计**，按时间回放全部日志、对四类规则逐条匹配并去重，输出结构化报告。

六、 系统流程图

为满足“流程图必须与实现对应、包含判断条件、输入输出和异常路径”的要求，本节给出四张流程图：系统总体流程图、登录认证流程图（核心）、登录失败限制流程图与异常登录审计流程图。每个判断分支均对应具体代码函数，详见图下说明。

6.1 系统总体流程图

图 1 给出系统的总体处理流程，对应 `app.py` 的路由分发。

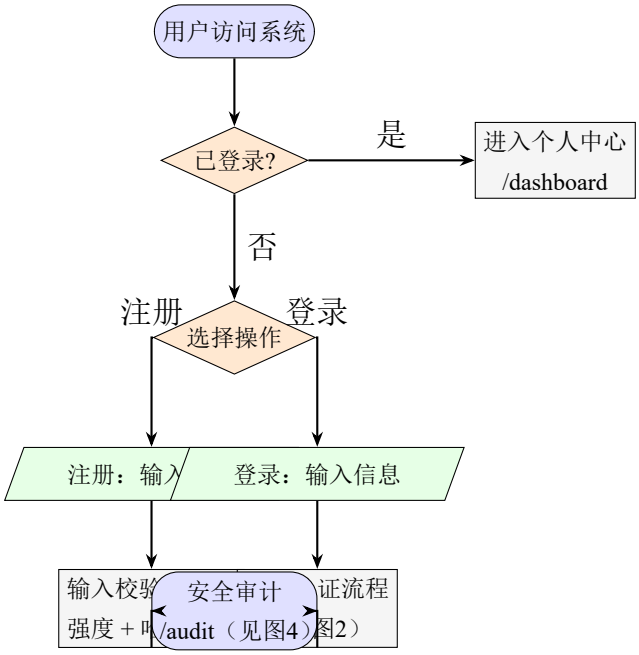


图 **1**　系统总体流程图

6.2 用户登录认证流程图（核心）

图 2 对应 `security.authenticate()` 函数，完整体现“查询用户 → 判断锁定 → 判断用户是否存在 → 校验口令 → 成功后实时异常检测”的判断分支与异常路径。

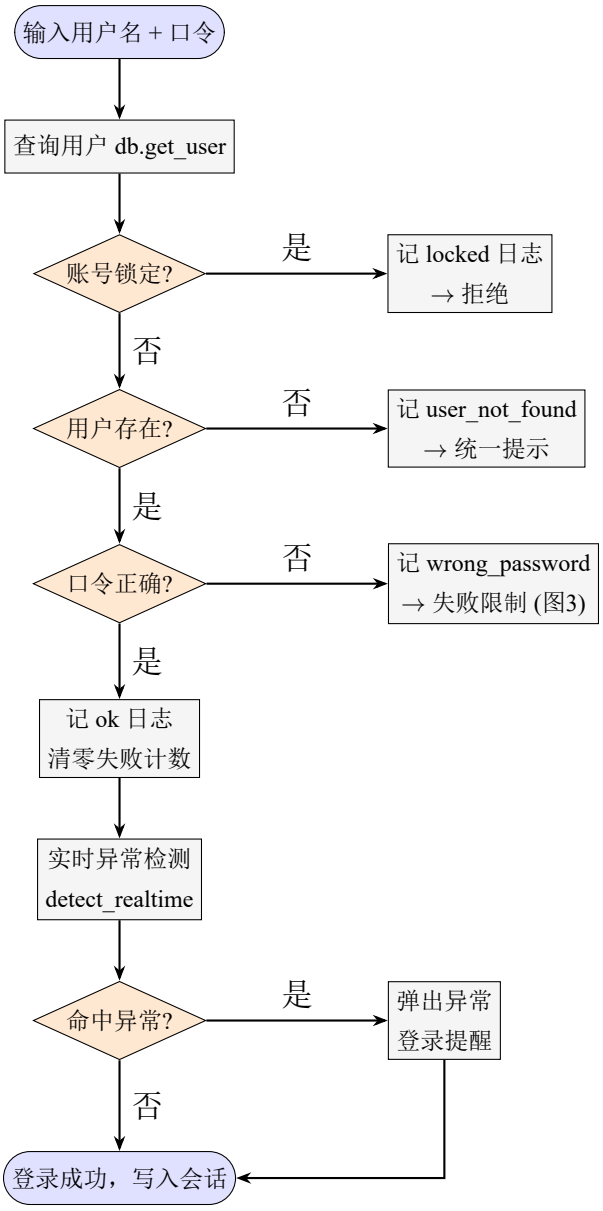


图 **2**　用户登录认证流程图

6.3 登录失败限制流程图

图 3 对应口令错误分支与 `count_recent_failures()`：失败计数加一、判断阈值、锁定账号、记录告警。

6.4 异常登录审计流程图

图 4 对应 `audit.run_full_audit()`：清空旧告警、按时间遍历日志、对四类规则匹配、汇总输出审计结果。

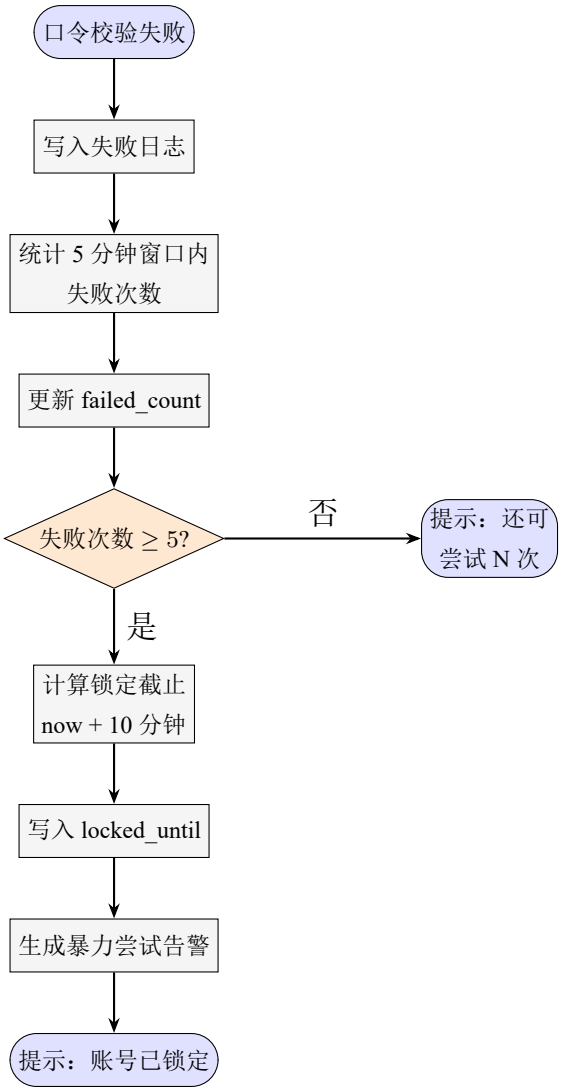


图 **3**　登录失败限制流程图

七、 系统实现

7.1 登录认证核心

登录认证严格按图 2 的四个分支实现：账号锁定则直接拒绝；用户不存在返回统一提示防枚举；口令错误进入失败限制流程；认证成功后清零计数并运行实时异常检测。核心逻辑如代码清单 3 所示。

```
def authenticate(username, password, ip, user_agent):
    with db.get_conn() as conn:
        user = db.get_user(conn, username)
        # branch 1: account locked -> reject directly
        if user is not None and is_locked(user):
            db.add_login_log(conn, username, ip, user_agent, False, "locked", ts)
            return {"success": False, "reason": "locked", ...}
        # branch 2: user not found -> unified message (anti-enumeration)
```

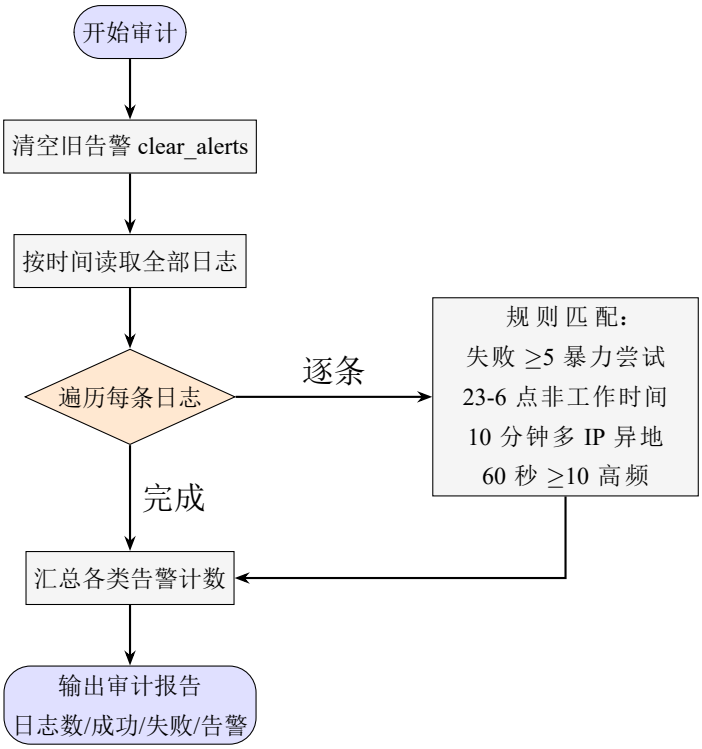


图 **4**　异常登录审计流程图

```
    if user is None:
        db.add_login_log(conn, username, ip, user_agent, False, "user_not_found", ts)
        return {"success": False, "reason": "user_not_found", ...}
    # branch 3: wrong password -> count + maybe lock
    if not verify_password(user["password_hash"], password):
        db.add_login_log(conn, username, ip, user_agent, False, "wrong_password", ts)
        failures = count_recent_failures(conn, username)
        if failures >= MAX_FAILED_ATTEMPTS:
            db.set_lock(conn, username, lock_until) # lock 10 min
            db.add_alert(conn, username, "brute_force", ...)
        return {"success": False, "reason": "wrong_password", ...}
    # branch 4: success -> reset + realtime anomaly detection
    db.add_login_log(conn, username, ip, user_agent, True, "ok", ts)
    db.reset_user_security(conn, username, ts)
    alerts = audit.detect_realtime(conn, username, ip, now())
return {"success": True, "reason": "ok", "alerts": alerts}
```

Listing 3: 登录认证主流程（security.authenticate 节选）

7.2 异常检测实现

非工作时间判断需处理跨午夜区间（23:00 至次日 06:00），异地登录通过统计窗口内不同 IP 数实现，高频登录通过秒级窗口计数实现，如代码清单 4 所示。

```python
def is_off_hours(dt):          # rule 3: cross-midnight interval
    h = dt.hour
    start, end = 23, 6
    return h >= start or h < end

def distinct_ips_in_window(conn, username, end_dt, minutes): # rule 2
    start = (end_dt - timedelta(minutes=minutes)).isoformat(sep=" ")
    cur = conn.execute("SELECT DISTINCT ip FROM login_logs "
       "WHERE username=? AND created_at>=? AND created_at<=?",
       (username, start, end_dt.isoformat(sep=" ")))
    return [r["ip"] for r in cur.fetchall()]
```

Listing 4: 异常规则判断（audit.py 节选）

八、 实验数据

8.1 测试账号（本人创建）

本人在 `seed_data.py` 中创建 4 个测试账号，口令均经加盐哈希存储，如表 4 所示。

表 **4**　本人创建的测试账号

用户名	用途	口令（已加盐哈希存储）
czk2023	主账号（程泽凯本人）	Czk@2023112621
alice99	普通用户	Alice#pass01
bob_dev	普通用户	BobDev2024!
victim7	演示被暴力破解的受害账号	Victim#0001

口令哈希的真实存储样例（库中无任何明文）：

```
czk2023 -> pbkdf2:sha256:260000$W10OrlyPNRYsjpci$8bfdef18688201d9f...
alice99 -> pbkdf2:sha256:260000$77QdBCxQwoKr8Zoy$01689276b56190d39...
```

8.2 模拟异常登录记录（本人设计）

为覆盖四类异常并保留一组正常对照，本人设计了如表 5 所示的模拟数据。

表 **5**　本人设计的模拟登录场景

场景	账号	设计的数据
A 暴力尝试	victim7	3 分钟内连续失败 6 次
B 异地登录	alice99	4 分钟内来自 10.0.0.5 / 172.16.8.9 / 203.0.113.7
C 非工作时间	bob_dev	凌晨 02:30 成功登录
D 高频登录	czk2023	40 秒内尝试 12 次
E 正常对照	czk2023	白天 14:00 单 IP 低频登录（不应告警）

8.3 审计真实输出

对上述种子数据运行全量审计，本机真实输出为：日志总数 23 条，其中成功登录 6 条、失败登录 17 条，共生成 8 条告警，分类统计如表 6 所示。

表 **6**　审计告警分类统计（本机真实结果）

异常类型	命中次数
暴力尝试	2
异地登录	2
非工作时间登录	1
高频登录	3
合计	8

部分告警明细（真实截取）：

```
[非工作时间登录] bob_dev 于 2026-05-26 02:30:00 在非工作时段成功登录
[暴力尝试]     victim7 截至 19:14:26, 5 分钟内失败 5 次
[异地登录]     alice99 10 分钟内出现 3 个 IP: 10.0.0.5, 172.16.8.9, 203.0.113.7
[高频登录]     czk2023 60 秒内请求 12 次
```

九、 测试用例与运行结果

9.1 测试用例

自动化测试脚本 `run_tests.py` 中每个用例独立重建数据库以保证隔离，共 8 条用例、18 项断言，覆盖不同类型，如表 7 所示。

表 **7**　测试用例（≥ 5 条，覆盖不同类型）

编号	测试用例	类型	结果
TC1	注册 + 口令哈希（无明文/pbkdf2 前缀/弱口令被拒）	安全存储	通过
TC2	正确口令登录成功	正常功能	通过
TC3	连续失败 5 次触发账号锁定	失败限制	通过
TC4	锁定期内正确口令仍被拒绝	失败限制	通过
TC5	异地登录检测（短时多 IP）	异常检测	通过
TC6	非工作时间登录检测	异常检测	通过
TC7	高频登录检测	异常检测	通过
TC8	全量审计汇总（覆盖四类异常）	审计	通过

9.2 运行结果

自动化测试最终输出 **18/18 项断言通过**。此外，通过 Flask 测试客户端进行冒烟测试，`/login`、`/register`、`/audit`、`/dashboard` 均返回状态码 200，注册 → 登录 → 个人中心流程完整贯通，说明系统功能正确、流程闭环。

十、 问题与改进

10.1 遇到的问题及解决

1. **时间窗口统计：**最初使用简单计数器累加失败次数，无法表达“5 分钟内”的时间语义；改为基于登录日志按时间窗口统计后，时间约束变得精确。
2. **用户枚举风险：**用户不存在与口令错误若返回不同提示，会泄露账号是否存在；统一为“用户名或口令错误”后修复该信息泄露问题，符合 OWASP Top 10 中对敏感信息暴露的防护建议 [3]。

3. **审计可重复性**：重复审计会重复写入告警；通过审计前 `clear_alerts` 加去重集合解决，保证幂等。

10.2 改进方向

- 引入图形验证码或基于时间的一次性口令（TOTP）二次验证；
- 接入 IP 地理库实现真正的异地判定（当前为同账号多 IP 启发式）；
- 接入限流中间件，对高频来源 IP 自动限流；
- 增加告警推送（邮件 / Webhook）并接入集中式日志分析（如 ELK）。

10.3 答辩可现场验证的参数

修改 `config.py` 中任一阈值后重新运行 `python run_tests.py` 或刷新 `/audit` 即可验证效果，例如：将 `MAX_FAILED_ATTEMPTS` 由 5 改为 3，账号锁定将更早触发；将 `HIGH_FREQ_THRESHOLD` 由 10 改为 5，高频告警数量随之上升。

十一、 总结

本课程设计完整实现了一个具备安全加固与审计能力的 Web 登录系统，覆盖任务书要求的全部 7 项必做功能与 4 类异常规则，做到了“有实现、有数据、有流程图、有验证”。系统以 PBKDF2 加盐慢哈希存储口令，对失败登录进行基于时间窗口的精确计数并自动锁定账号，记录完整的登录审计日志，并通过实时检测与离线全量审计识别暴力尝试、异地登录、非工作时间登录与高频登录四类异常。最终 18/18 项自动化断言全部通过，四张流程图与代码分支一一对应，全部安全阈值由本人设计且集中可配，答辩时可现场修改参数重新验证。通过本设计，本人加深了对 Web 认证安全、口令存储、登录审计与异常行为检测等网络空间安全核心技术的理解与实践能力。

参考文献

[1] Paul A. Grassi, James L. Fenton, and Elaine M. Newton. Digital identity guidelines: Authentication and lifecycle management (sp 800-63b). Technical Report SP 800-63B, National Institute of Standards and Technology (NIST), 2020.

[2] Kathleen Moriarty, Burt Kaliski, and Andreas Rusch. Pkcs #5: Password-based cryptography specification version 2.1 (rfc 8018). `https://www.rfc-editor.org/rfc/rfc8018`, 2017.

[3] OWASP Foundation. Owasp top 10:2021 – the ten most critical web application security risks. https://owasp.org/Top10/, 2021.

[4] OWASP Foundation. Owasp authentication cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html, 2023.